Threat Model for ViperVault

Emanuele Relmi

Abstract

This document presents a comprehensive threat model for **ViperVault**, a password-manager application currently deployed as a native mobile client for Android and iOS. The immediate goal of this model is to assess the risks that arise in the current *local-only* configuration of the application, where all data is stored and encrypted directly on the user's device, with no cloud interaction. At the same time, the model anticipates the project's long-term roadmap, which includes the release of desktop clients (Windows, macOS, Linux), browser extensions and the introduction of a cloud-based synchronization and update service. To provide a structured and reproducible analysis, the identified threats are categorized using the **STRIDE** methodology, ensuring full alignment with industry standards in threat modeling. Each threat is described in terms of adversarial action, assessed by likelihood and impact and paired with concrete mitigations that are feasible within the project's constraints. This approach makes it possible to distinguish between immediate device-level risks, such as clipboard hijacking or malware infection and more advanced future threats, such as man-in-the-middle attacks on synchronization, supply-chain compromises or state-sponsored adversaries targeting the backend infrastructure. The threat model is further contextualized against major security and privacy frameworks, including **OWASP**, **NIST SP 800-63** and European regulatory standards such as **GDPR** and **NIS2**. By embedding security- and privacy-by-design principles throughout the analysis, ViperVault establishes not only a defensive baseline for the present, but also a forward-looking foundation for sustainable and compliant growth. In summary, this document serves both as a snapshot of the current security posture of ViperVault and as a living reference for guiding the project through its future evolution, ensuring that usability, security and regulatory compliance remain in balance at every stage.

Contents

1 Scope and Assumptions

The scope of this threat model is defined around the current and future architecture of **ViperVault**. At present, the application is deployed as a local-only installation on Android and iOS devices. All credential data is stored directly on the device and protected through strong cryptographic mechanisms. There is no network communication and no persistent cloud storage, which significantly reduces the remote attack surface. Looking ahead, however, the project's roadmap includes several major extensions: the release of desktop clients for Windows, macOS and Linux; the development of browser extensions to facilitate password autofill; the addition of a cloud synchronization service to enable seamless access across devices. These planned features will necessarily expand the system boundaries and introduce new classes of threats that must be considered from the outset. In terms of adversaries, the model assumes a broad spectrum of potential threat actors. These include opportunistic attackers exploiting common weaknesses, more sophisticated advanced persistent threats (APTs) conducting targeted campaigns, malicious insiders with privileged access and supply-chain adversaries attempting to compromise the build or distribution process. The security goals of ViperVault are fourfold. First, the confidentiality of all stored credentials must be preserved under every circumstance. Second, the integrity of both the vault and its metadata must be ensured, preventing unauthorized tampering. Third, the application must remain reliably available to legitimate users. Finally, non-repudiation must be enforced for critical operations such as master-password changes, ensuring accountability for sensitive security events. The cryptographic foundations of the system rest on well-established and modern primitives. The vault is encrypted using the XChaCha20-Poly1305 algorithm, which offers strong authenticated encryption guarantees. The master password is processed through Argon2id, a memory-hard key derivation function designed to resist GPU and ASIC cracking attempts. All keys are derived exclusively on the user's device and are never transmitted, in line with zero-knowledge design principles.

2 Alignment with Standards and Regulations

Security and privacy requirements do not exist in isolation: they are guided by community best practices, technical standards and legal frameworks. This section places ViperVault in relation to the most relevant reference points, namely the OWASP guidelines, the NIST recommendations and European regulations such as GDPR and NIS2. By aligning the threat model with these standards, the project demonstrates not only technical robustness but also regulatory compliance and long-term sustainability.

2.1 OWASP and NIST

ViperVault has been designed with explicit reference to the **OWASP Mobile Application Security Verification Standard (MASVS)** and the **Mobile Security Testing Guide (MSTG)**. Core requirements such as secure credential storage (MASVS-STORAGE-2), strong and up-to-date cryptography, secure input handling and the wiping of sensitive information from memory are explicitly addressed. This ensures that the application does not merely rely on cryptographic primitives but also respects best practices for their implementation and for the surrounding environment. In addition, the application complies with recommendations from **NIST SP 800-63B** on digital authentication. Specifically, the use of **Argon2id** as a memory-hard key derivation function satisfies the requirement to resist large-scale GPU or ASIC cracking attempts, while the password policy enforces sufficient entropy, prohibits commonly used secrets and mitigates risks associated with password reuse. Together, these controls position ViperVault in line with modern expectations for secure identity and credential management.

2.2 GDPR

From a legal and privacy perspective, the design of ViperVault is strongly aligned with the **General Data Protection Regulation (GDPR)**. The application follows the principle of **Privacy by Design**, meaning that it collects and processes only the minimum data required for its function. All vault contents are encrypted client-side, no personal identifiers are stored in plaintext and no unnecessary telemetry or logs are generated. The **right to erasure** is also inherently respected: once a vault is deleted, it is cryptographically unrecoverable, ensuring that the user retains full control over their data lifecycle. Furthermore, the principle of **data minimization** is applied rigorously, with no additional metadata (such as usage statistics or access logs) being collected. This not only protects users from privacy risks but also simplifies compliance with regulatory audits.

2.3 NIS2

Furthermore, ViperVault is compliant with the European **NIS2 Directive**, which expands cybersecurity obligations for digital service providers. Even though the project is currently developed by a single developer, mechanisms are planned to support a security management lifecycle. These include **reproducible builds** to ensure the integrity of released binaries, a process for responsible **vulnerability disclosure** and the use of **digitally signed updates** to prevent tampering. In terms of incident response, the project prioritizes transparency and agility. Public transparency logs ensure that update integrity can be independently verified and a lightweight process for rapid patching helps minimize exposure when new vulnerabilities are discovered. In this way, even with limited resources, ViperVault can embody the spirit of NIS2: resilience, accountability and continuous improvement in security.

3 Threat Taxonomy (STRIDE)

3.1 Spoofing

Threat	Description	Likelihood	Impact	Mitigations
Phishing / Fake App	User tricked into revealing master-password or installing malicious replica.	Medium	High	Code signing, app store integrity, MFA
Biometric Spoofing	Synthetic fingerprint/face sample bypass sensors.	Low	High	Secure enclave validation, liveness detection
WebAuthn Replay	Captured assertion reused.	Low	High	Nonce-based challenges, secure enclave keys

3.2 Tampering

Threat	Description	Likelihood	Impact	Mitigations
Vault File Tampering	Malicious modification of local vault file.	Low	High	Integrity checks, authenticated encryption
DLL / Library Hijacking	Malicious libraries injected.	Low	Medium	Signed libraries, secure load paths
MITM on Sync	Intercept TLS traffic in future sync.	Low (future)	High	TLS 1.3 + certificate pinning
Transparency-Log Forgery	Tampered update manifests.	Low	Critical	Transparency logs, signed manifests

3.3 Repudiation

Threat	Description	Likelihood	Impact	Mitigations
Replay of Reset Tokens	Token reused to regain access.	Low	High	Expiry, signed tokens, vault re-encryption

3.4 Information Disclosure

Threat	Description	Likelihood	Impact	Mitigations
Clipboard Hijacking	Malicious app reads clipboard.	Medium	Medium	Auto-clear clipboard, warnings
Memory Dump / RAM Scraping	Keys extracted from memory.	Low	High	Secure enclave, memory wipe
Unencrypted Backup	Vault stored without encryption.	Medium	High	Force client-side encryption
XSS in Extension	Malicious script steals data.	Low (future)	High	Strict CSP, sandboxed extensions
Side-Channel (Timing, Spectre)	Infer data via timing or speculative execution.	Low	High	Constant-time KDF, patched OS

3.5 Denial of Service

Threat	Description	Likelihood	Impact	Mitigations
Brute Force Lockout	Automated guessing attempts.	Medium	High	KDF cost, rate-limiting
Vault Write Races	Concurrent edits corrupt vault.	Medium	High	File locking, transactional writes
Device Theft or Loss	Loss of unlocked device.	Medium	High	Auto-lock, remote wipe
Overlay / Accessibility DoS	Malicious overlay prevents input.	Low	Medium	Overlay detection, secure flags

3.6 Elevation of Privilege

Threat	Description	Likelihood	Impact	Mitigations
Privilege Escalation	Exploit escape from sandbox.	Low	High	OS hardening, least privilege
Supply-Chain Compromise	Malicious dependency injected.	Low	High	Reproducible builds, signed updates
State-Sponsored APT	Highly resourced adversary.	Low (future)	Critical	Post-quantum crypto, strict E2EE

4 Proposed Improvements

The following enhancements are proposed to strengthen ViperVault's security posture and to anticipate risks that may emerge as the application evolves. Each improvement is consistent with the principles of security-by-design and privacy-by-design, while remaining feasible for a project developed with limited resources.

- **Duress / Panic Mode**: Introduce an optional mechanism that allows users to configure either a decoy vault or a dedicated "duress password." This feature is intended for scenarios where a user is coerced into unlocking the application. Under such conditions, entering the duress password would open a harmless decoy vault, thereby protecting the true sensitive data.

- **Master Password Reset**: Reinforce the design principle that the master password cannot be trivially reset. In case of loss, the vault should remain inaccessible, which aligns with the zero-knowledge model. To provide a safety net, recovery codes could optionally be generated at first setup and securely stored offline by the user, ensuring that recovery remains possible without weakening overall security.

- **Accessibility Overlay Protection**: Implement controls to detect suspicious screen overlays or accessibility abuse. These overlays can be exploited to trick users into interacting with spoofed interfaces. By actively monitoring and blocking untrusted overlays, ViperVault can reduce the risk of visual spoofing and fraudulent input capture.

- **Secure Notifications**: Ensure that system notifications never reveal sensitive information. Instead of displaying usernames, vault labels or credential data, notifications should carry only opaque tokens or neutral text, thereby avoiding accidental data leakage through the operating system's notification channels.

- **Anti-Forensics**: Strengthen resistance against forensic attempts by applying multiple safeguards. These include automatic memory wiping when the app is backgrounded, periodic clearing of the clipboard after use and anti-debugging checks that make it harder for adversaries to inspect the application during runtime.

5 Future Enhancements: Zero-Knowledge Proofs

Zero-Knowledge Proofs (ZKP) are not required in the current local-only architecture of ViperVault, since no server-side authentication or interaction exists. However, once a synchronization backend and OTA update server are introduced, ZKP-based protocols can provide strong security and privacy advantages.

5.1 Potential Use Cases

- **Server Authentication (Sync/Login)**: Instead of sending password-derived hashes to the server, ViperVault can implement a Password-Authenticated Key Exchange (PAKE) with ZK properties, such as OPAQUE. This allows the client to prove knowledge of the master password without revealing it or any verifiable hash to the server.
- **Vault Integrity Proofs**: Clients could prove that uploaded vaults are correctly encrypted under the right key, without disclosing the key itself.

5.2 Advantages

- No password hashes or secrets ever stored on the server.
- Protection against offline brute-force in case of server breach.
- Strong alignment with Zero-Knowledge Architecture principles and GDPR/NIS2 minimization of personal data processing.
- Clear FOSS trust signal: the server cannot know secrets even in hashed form.

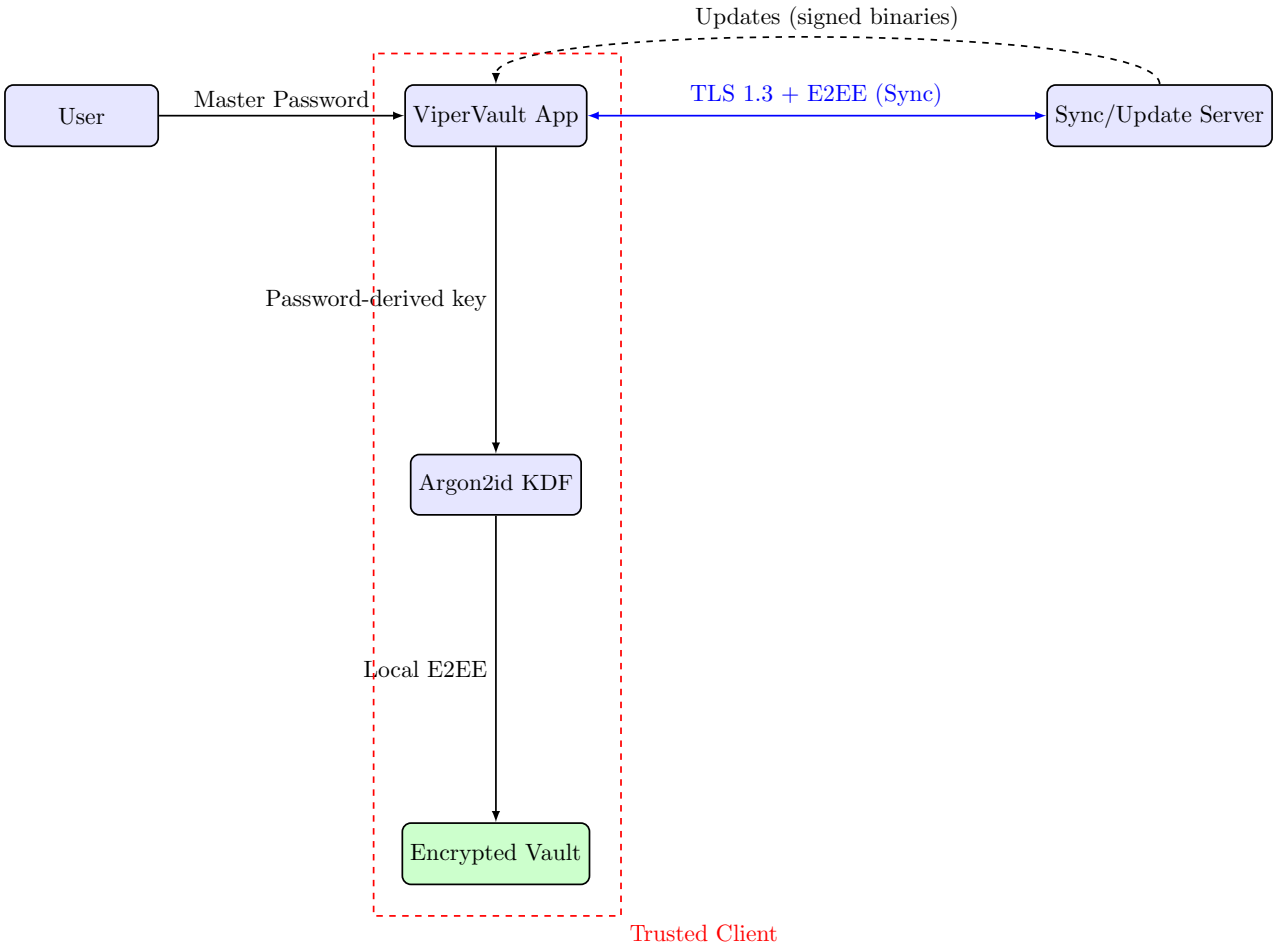
5.3 Disadvantages

- Increased implementation complexity, requiring reliable cryptographic libraries and careful protocol design.
- Higher performance cost than traditional logins, especially on low-end devices.

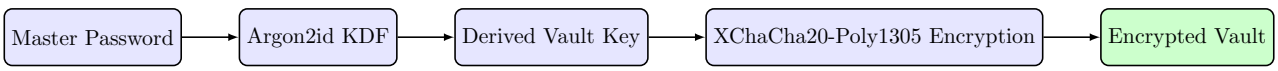
6 Diagrams

6.1 Current Architecture (Local-Only)

6.2 Future Architecture (Cloud Sync + Updates)



6.3 Encryption Flow



7 Conclusion

The threat model developed for ViperVault demonstrates a strong alignment with established security and privacy standards, including OWASP, NIST, GDPR and the NIS2 directive. At its current stage of development, where the application operates entirely in local-only mode, the most pressing risks are those related to the end-user device itself. Examples include the presence of local malware, clipboard hijacking or physical device theft. These are threats that can be effectively mitigated with secure enclave usage, automatic vault locking and secure clipboard handling. Looking ahead, the planned evolution of ViperVault towards a synchronization server and browser extensions introduces a more complex threat landscape. In these future scenarios, the attack surface broadens and more sophisticated adversaries may come into play, such as supply-chain compromises, advanced persistent threats or side-channel attacks. Addressing these challenges from the design phase is essential in order to ensure that the transition to a multi-platform, cloud-enabled ecosystem does not erode the strong guarantees already present in the current mobile app. The proposed improvements — such as the introduction of a duress or panic mode, the design of a secure and non-recoverable master password reset mechanism and anti-forensic measures like memory wiping and overlay detection — further strengthen the defensive posture of the project. Combined with ViperVault's commitment to security- and privacy-by-design principles, these measures provide a clear roadmap for maintaining both user trust and regulatory compliance as the application grows in scope and functionality. In conclusion, ViperVault's threat model not only addresses the risks of the present but also anticipates those of the future, laying down a foundation for a robust, resilient and trustworthy password management solution.